NEXT ENGINEER · _SHEERAT_

Compiler Design

BCS602

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE | **Semester:** 6 | **Credits:** 4 | **L-T-P:** 3-1-0

Scheme: AKTU B.Tech 3rd year, Core, 2024-25 onwards

UNIT 1 Introduction to Compiler, Lexical Analysis and Grammars

UNIT 2 Basic Parsing Techniques

UNIT 3 Syntax-Directed Translation and Intermediate Code

UNIT 4 Symbol Tables, Run-Time Administration and Error Handling

UNIT 5 Code Generation and Code Optimization

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: phases, algorithms and comparison tables.
3. **Solved examples**: the numericals that appear every year (regular expression to DFA, FIRST and FOLLOW, LL(1) table, SLR/CLR/LALR tables, three-address code, quadruples and triples, basic blocks, DAG).
4. **Formulas / key points to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (phases with the example `position = initial + rate * 60`, bootstrapping, RE to DFA, LEX, ambiguity) and Unit 2 (left recursion, left factoring, FIRST/FOLLOW, LL(1), operator precedence, LR(0) items, SLR, CLR, LALR).
- **Afternoon (3 hours):** Unit 3 (SDD and SDT, postfix, syntax tree, three-address code, quadruples, triples, Boolean and control-flow translation) and Unit 4 (symbol table, activation records, storage allocation, error recovery).
- **Evening (2 hours):** Unit 5 (basic blocks, flow graphs, DAG, loop optimization, peephole, data-flow equations, code generator). Then the "Last-hour checklist".
- **In the exam:** parsing table questions carry 7 to 10 marks. Write the augmented grammar, the item sets, the GOTO graph and the table, and then parse the given string with a stack trace. That full sequence earns full marks.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to Compiler, Lexical Analysis and Grammars

✍️ 2-mark definitions

- **Compiler:** a program that translates a program in a source language (high level) into an equivalent program in a target language (machine or assembly code), reporting errors.
- **Interpreter:** executes the source program statement by statement without producing a separate target program.
- **Phase:** a logically separate step of compilation (lexical analysis, syntax analysis, and so on).
- **Pass:** one complete reading of the source (or intermediate) program. Several phases can be grouped into one pass.
- **Front end and back end:** the front end (analysis: lexical, syntax, semantic analysis, intermediate code) depends on the source language; the back end (synthesis: optimization, code generation) depends on the target machine.
- **Cross compiler:** a compiler that runs on one machine and produces code for another.
- **Bootstrapping:** writing a compiler in the language it compiles (or a subset), and using simpler compilers to build it step by step.
- **Token:** a category of lexical unit such as id, number, keyword, operator.
- **Lexeme:** the actual character sequence that matches a token (for example `count` for token id).
- **Pattern:** the rule (usually a regular expression) describing the lexemes of a token.
- **Regular expression:** a notation for regular languages built from symbols with union (|), concatenation and Kleene star (*).
- **Finite automaton:** a recognizer for a regular language; NFA (non-deterministic) or DFA (deterministic).
- **Context-free grammar (CFG):** G = (V, T, P, S) with variables V, terminals T, productions P of the form A → α, and start symbol S.
- **Ambiguous grammar:** a grammar in which some string has more than one parse tree (or more than one leftmost derivation).
- **BNF (Backus-Naur Form):** a notation for writing CFGs, for example `<expr> ::= <expr> + <term> | <term>`.
- **LEX and YACC:** LEX generates a lexical analyzer from regular expressions; YACC (Yet Another Compiler-Compiler) generates an LALR(1) parser from a grammar.

💡 Core concepts

Phases of a compiler (with the classic example `position = initial + rate * 60`)

Phase	Output for the example
Lexical analysis	`<id,1> <=> <id,2> <+> <id,3> <*> <60>`
Syntax analysis	Parse/syntax tree: `=` at the root with left child id1 and right child `+(id2, *(id3, 60))`

Phase	Output for the example
Semantic analysis	Type checking; inserts `inttofloat(60)` because rate is float
Intermediate code generation	`t1 = inttofloat(60)`; `t2 = id3 * t1`; `t3 = id2 + t2`; `id1 = t3`
Code optimization	`t1 = id3 * 60.0`; `id1 = id2 + t1`
Code generation	`LDF R2, id3`; `MULF R2, R2, #60.0`; `LDF R1, id2`; `ADDF R1, R1, R2`; `STF id1, R1`

The **symbol table manager** and the **error handler** interact with all phases.

Language processing system: source program → preprocessor (macros, file inclusion) → compiler → assembler → relocatable machine code → linker/loader (with library files) → executable.

Single-pass vs multi-pass: a single-pass compiler is faster and needs less memory but limits optimization and needs declarations before use; a multi-pass compiler uses more memory but allows better optimization and portability.

Bootstrapping (T-diagrams)

- A compiler is shown as a T with source language S, target language T and implementation language I, written S_I^T.
- **Example:** to build a compiler for a new language L on machine A: write a compiler for a small subset of L in assembly of A, then write the full L compiler in that subset and compile it with the first compiler. To get a compiler for machine B (cross compiler), write L→B in L and compile it with the existing L compiler on A, which gives L→B running on A; compiling the same source again with that gives L→B running on B.

Lexical analysis

- Role: read characters, group them into lexemes, produce tokens, remove white space and comments, keep line numbers for error messages, enter identifiers in the symbol table.
- **Input buffering:** two-buffer scheme with lexemeBegin and forward pointers; sentinels (eof) at the end of each buffer half reduce the tests per character to one.
- **Regular expression → NFA (Thompson's construction) → DFA (subset construction) → minimized DFA:** the standard path for building a lexer automatically.
- **Optimization of DFA-based pattern matchers:** build the DFA directly from the regular expression using the syntax tree with nullable, firstpos, lastpos and followpos functions (no NFA needed); minimize states by partitioning into accepting and non-accepting groups and refining; compress transition tables.

LEX program structure

NEXT ENGINEER _sheerat_

```
%{
/* declarations: C code, e.g. #include <stdio.h> */
int count = 0;
%}
digit   [0-9]
letter  [a-zA-Z]
%%
{letter}({letter}|{digit})*   { count++; printf("ID: %s\n", yytext); }
{digit}+                      { printf("NUM: %s\n", yytext); }
[ \t\n]                       ;   /* skip white space */
.                             { printf("OTHER: %s\n", yytext); }
%%
int main(void) { yylex(); printf("identifiers = %d\n", count); return 0; }
int yywrap(void) { return 1; }
```

Parts: declarations, `%%` translation rules (pattern { action }), `%%` auxiliary functions. `yytext` holds the lexeme, `yyleng` its length, `yylex()` returns the next token. Conflict rules: the longest match wins; for equal length, the rule listed first wins.

YACC program structure: declarations (`%token` , `%left` , `%right` for precedence), `%%` grammar rules with actions (`$$` is the value of the left side, `$1` , `$2` are values of right-side symbols), `%%` supporting C routines. YACC builds an LALR(1) table and resolves shift-reduce conflicts by precedence and associativity (default: shift) and reduce-reduce conflicts by choosing the earlier production.

Context-free grammars

- **Derivation:** leftmost (replace the leftmost variable each step) or rightmost (canonical).
- **Parse tree:** root = start symbol, interior nodes = variables, leaves = terminals or ε; yield = the string derived.
- **Capabilities of CFG:** CFGs describe nested structures (balanced parentheses, nested blocks, expressions with precedence) that regular expressions cannot. They cannot express some context-sensitive checks, such as "variable declared before use" or "number of arguments equals number of parameters"; these go to semantic analysis.
- **Removing ambiguity:** introduce precedence levels (E → E + T | T; T → T * F | F; F → (E) | id) and associativity (left recursion for left-associative operators).

🧮 Solved examples

Example 1 (tokens). Count the tokens in `printf("i = %d, &i = %x", i, &i);` Tokens: `printf` , `(` , `"i = %d, &i = %x"` , `,` , `i` , `,` , `&` , `i` , `)` , `;` → **10 tokens**.

Example 2 (ambiguity). Grammar E → E + E | E * E | id; string `id + id * id` .

- Tree 1: E → E + E with the right E → E * E (multiplication done first; the correct meaning).
- Tree 2: E → E * E with the left E → E + E (addition done first).
- Two parse trees, so the grammar is **ambiguous**. The unambiguous version: E → E + T | T, T → T * F | F, F → (E) | id.

Example 3 (regular expression to minimized DFA). For (a|b)*abb, the minimized DFA has 4 states (A start, D accepting):

State	a	b
→ A	B	A
B	B	C
C	B	D
* D	B	A

Check `aabb`: A -a→ B -a→ B -b→ C -b→ D (accepted). Check `abab`: A→B→C→B→C (not accepting, rejected).

Example 4 (subset construction idea). ε-closure(0) of Thompson's NFA for (a|b)*abb is {0, 1, 2, 4, 7} = A. Move on a gives {3, 8}; its ε-closure {1, 2, 3, 4, 6, 7, 8} = B. Continue until no new sets: 5 DFA states A to E, and minimization merges A and C to give the 4-state DFA above.

Key points to remember

- Six phases + symbol table + error handler. Analysis (front end) vs synthesis (back end).
- Token / lexeme / pattern.
- RE → NFA (Thompson) → DFA (subset) → minimize. Direct method: followpos.
- LEX: longest match, then first rule. YACC: LALR(1), default shift.
- Ambiguity: two parse trees; remove by precedence and associativity.

⭐ Important questions

1. Explain the phases of a compiler with a diagram, and show the output of each phase for `position = initial + rate * 60`.
2. Differentiate compiler and interpreter, phase and pass, single-pass and multi-pass compilers.
3. What is bootstrapping? Explain with T-diagrams how to build a cross compiler.
4. Explain the role of the lexical analyzer, input buffering with sentinels, and the structure of a LEX program.
5. Convert a regular expression to an NFA and then to a minimized DFA.
6. Define CFG, derivation, parse tree and ambiguity. Show that a given grammar is ambiguous and remove the ambiguity.

UNIT 2 Basic Parsing Techniques

✍️ 2-mark definitions

- **Parser (syntax analyzer):** takes the token stream and checks that it can be generated by the grammar, building a parse tree or reporting errors.
- **Top-down parsing:** builds the parse tree from the root to the leaves, finding a leftmost derivation (recursive descent, predictive LL(1)).
- **Bottom-up parsing:** builds the tree from the leaves to the root, finding a rightmost derivation in reverse (shift-reduce, operator precedence, LR).
- **Left recursion:** a production A → Aα. It makes top-down parsers loop forever.
- **Left factoring:** rewriting productions with a common prefix (A → αβ1 | αβ2) as A → αA', A' → β1 | β2, so a predictive parser can choose.
- **FIRST(α):** the set of terminals that can begin strings derived from α (including ε if α ⇒* ε).
- **FOLLOW(A):** the set of terminals that can appear immediately to the right of A in some sentential form ($ is in FOLLOW(start)).
- **LL(1) grammar:** a grammar that can be parsed top-down, scanning Left to right, producing a Leftmost derivation, with 1 token of lookahead; its predictive table has no multiple entries.
- **Handle:** a substring that matches the right side of a production and whose reduction is one step of the reverse rightmost derivation.
- **Viable prefix:** a prefix of a right-sentential form that can appear on the stack of a shift-reduce parser.
- **LR(0) item:** a production with a dot in the right side showing how much has been seen, for example A → X·YZ.
- **LR(1) item:** an LR(0) item with a lookahead terminal, [A → α·β, a].
- **Shift-reduce conflict / reduce-reduce conflict:** a table cell where the parser cannot decide between shifting and reducing, or between two reductions.
- **Operator grammar:** a grammar with no ε-productions and no two adjacent variables on any right side.

💡 Core concepts

Removing left recursion: A → Aα | β becomes A → βA', A' → αA' | ε. Example: E → E + T | T becomes E → TE', E' → +TE' | ε.

Left factoring: S → iEtS | iEtSeS | a becomes S → iEtSS' | a, S' → eS | ε.

Rules for FIRST

1. If X is a terminal, FIRST(X) = {X}.
2. If X → ε, add ε.
3. If X → Y1 Y2 … Yk, add FIRST(Y1) − {ε}; if Y1 can derive ε, add FIRST(Y2) − {ε}, and so on; if all can derive ε, add ε.

Rules for FOLLOW

1. Put $ in FOLLOW(S).
2. For A → αBβ, add FIRST(β) − {ε} to FOLLOW(B).
3. For A → αB, or A → αBβ where β ⇒* ε, add FOLLOW(A) to FOLLOW(B).

Predictive (LL(1)) table construction: for each A → α: for each terminal a in FIRST(α), put A → α in M[A, a]; if ε is in FIRST(α), put A → α in M[A, b] for each b in FOLLOW(A) (including $). A grammar is LL(1) if no cell has two entries.

Recursive descent parser: one procedure per variable; each procedure matches terminals and calls procedures for variables. Without backtracking (predictive) it needs an LL(1) grammar.

Shift-reduce parsing: actions are shift (push the next input symbol), reduce (replace a handle on top of the stack by the left side), accept and error. Example with E → E + E | E * E | id and input id + id: shift id, reduce E → id, shift +, shift id, reduce E → id, reduce E → E + E, accept.

Operator precedence parsing

- Relations between terminals: a <· b (a yields precedence to b), a ≐ b (same precedence), a ·> b (a takes precedence).
- Rules: * has higher precedence than +, so + <· * and * ·> +. Left-associative operators: + ·> +, * ·> *. id ·> every operator, every operator <· id. $ <· everything and everything ·> $.
- Algorithm: if top <· input or top ≐ input, shift; if top ·> input, pop until the popped terminal is related by <· to the new top (reduce the handle).
- **Precedence functions** f and g can replace the table to save space (computed from a graph), but some error entries are lost.
- **Disadvantages:** works only for operator grammars; hard to handle unary minus; small class of grammars.

LR parsers

Parser	Items used	Lookahead in reduce	Power	Table size
LR(0)	LR(0) items	Reduce in every column	Weakest	Small
SLR(1)	LR(0) items	Reduce A → α only on FOLLOW(A)	More than LR(0)	Small
LALR(1)	LR(1) items, merging states with the same core	Merged lookaheads	Between SLR and CLR; used by YACC	Same number of states as SLR
CLR(1) / LR(1)	LR(1) items	Exact lookahead	Most powerful	Largest

Power: LR(0) ⊂ SLR(1) ⊂ LALR(1) ⊂ CLR(1). Merging CLR states into LALR can create reduce-reduce conflicts but never new shift-reduce conflicts.

LR parsing algorithm: stack of states; look at top state s and input a. ACTION[s, a] = shift j (push a and j), reduce A → β (pop 2|β| symbols, push A and GOTO[top, A]), accept, or error.

Closure and GOTO: CLOSURE(I) adds B → ·γ for every item A → α·Bβ in I (for LR(1), with lookahead FIRST(βa)). GOTO(I, X) = CLOSURE of items with the dot moved over X.

Using ambiguous grammars in LR parsing: an ambiguous expression grammar (E → E + E | E * E | (E) | id) gives a smaller table; conflicts are resolved with precedence and associativity rules (YACC's `%left`,

`%right`). The dangling-else conflict is resolved in favour of shift (else matches the nearest if).

Parser generator (YACC): input grammar with actions → y.tab.c (LALR parser) → compiled with the LEX-generated `yylex()` to produce a complete front end.

🧮 Solved examples

Example 1 (FIRST, FOLLOW and LL(1) table). Grammar after removing left recursion: E → TE'; E' → +TE' | ε; T → FT'; T' → *FT' | ε; F → (E) | id.

Variable	FIRST	FOLLOW
E	{ (, id }	{), $ }
E'	{ +, ε }	{), $ }
T	{ (, id }	{ +,), $ }
T'	{ *, ε }	{ +,), $ }
F	{ (, id }	{ +, *,), $ }

LL(1) table:

	id	+	*	(	)	$
E	E → TE'			E → TE'		
E'		E' → +TE'			E' → ε	E' → ε
T	T → FT'			T → FT'		
T'		T' → ε	T' → *FT'		T' → ε	T' → ε
F	F → id			F → (E)		

No cell has two entries, so the grammar is **LL(1)**.

Parse `id + id` :

Stack	Input	Action
$E	id+id$	E → TE'
$E'T	id+id$	T → FT'
$E'T'F	id+id$	F → id
$E'T'id	id+id$	match id
$E'T'	+id$	T' → ε
$E'	+id$	E' → +TE'
$E'T+	+id$	match +
$E'T	id$	T → FT'
$E'T'F	id$	F → id

Stack	Input	Action
$E'T'id	id$	match id
$E'T'	$	T' → ε
$E'	$	E' → ε
$	$	accept

Example 2 (SLR parsing table). Grammar: (0) S' → S, (1) S → CC, (2) C → cC, (3) C → d.

LR(0) item sets:

- I0: S' → ·S, S → ·CC, C → ·cC, C → ·d
- I1 = GOTO(I0, S): S' → S·
- I2 = GOTO(I0, C): S → C·C, C → ·cC, C → ·d
- I3 = GOTO(I0, c): C → c·C, C → ·cC, C → ·d (also GOTO(I2, c) and GOTO(I3, c))
- I4 = GOTO(I0, d): C → d· (also from I2 and I3 on d)
- I5 = GOTO(I2, C): S → CC·
- I6 = GOTO(I3, C): C → cC·

FOLLOW(S) = { $ }, FOLLOW(C) = { c, d, $ }.

State	c	d	$	S	C
0	s3	s4		1	2
1			accept		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

Parse `cdd`:

Stack	Input	Action
0	cdd$	shift 3
0 c 3	dd$	shift 4
0 c 3 d 4	d$	reduce C → d, GOTO(3, C) = 6
0 c 3 C 6	d$	reduce C → cC, GOTO(0, C) = 2
0 C 2	d$	shift 4
0 C 2 d 4	$	reduce C → d, GOTO(2, C) = 5
0 C 2 C 5	$	reduce S → CC, GOTO(0, S) = 1
0 S 1	$	accept

Example 3 (CLR and LALR for the same grammar). With LR(1) items there are **10 states**: the states for c·C, d· and cC· each appear twice, once with lookahead {c, d} (inside the first C) and once with lookahead {$} (inside the second C). In the usual numbering, I3/I6, I4/I7 and I8/I9 have the same cores. Merging them gives **7 LALR states** (I36, I47, I89), with the same table as SLR here and no conflicts.

Example 4 (operator precedence relation table) for id, +, *, $:

	id	+	*	$
id		·>	·>	·>
+	<·	·>	<·	·>
*	<·	·>	·>	·>
$	<·	<·	<·	accept

Parsing `id + id * id`: $ <· id ·> + → reduce id; then $ <· + <· id ·> * → reduce id; + <· * <· id ·> $ → reduce id; * ·> $ → reduce E * E; + ·> $ → reduce E + E; accept.

Formulas / key points to remember

- Left recursion A → Aα | β ⇒ A → βA', A' → αA' | ε.
- LL(1) condition: for A → α | β, FIRST(α) ∩ FIRST(β) = ø; and if β ⇒* ε, FIRST(α) ∩ FOLLOW(A) = ø.
- SLR: reduce on FOLLOW; CLR: reduce on the item's lookahead; LALR: merge same-core states.
- LR(0) ⊂ SLR ⊂ LALR ⊂ CLR. YACC = LALR(1).

⭐ Important questions

1. Eliminate left recursion and left factor a given grammar. Compute FIRST and FOLLOW and construct the LL(1) table; parse a string.
2. Explain shift-reduce parsing with handles and stack implementation. What conflicts can arise?
3. Explain operator precedence parsing with relations and precedence functions.
4. Construct the SLR parsing table for a given grammar and parse a string.
5. Construct CLR and LALR tables for S → CC, C → cC | d. Compare SLR, CLR and LALR.
6. How are ambiguous grammars used in LR parsing? Explain parser generators (YACC).

UNIT 3 Syntax-Directed Translation and Intermediate Code

✍️ 2-mark definitions

- **Syntax-directed definition (SDD):** a CFG with attributes attached to grammar symbols and semantic rules attached to productions.
- **Syntax-directed translation scheme (SDT):** a CFG with program fragments (semantic actions) embedded in the right sides of productions, showing the order of evaluation.
- **Synthesized attribute:** an attribute of a node computed from the attributes of its children (bottom-up).
- **Inherited attribute:** an attribute computed from the parent and siblings (top-down or sideways).
- **S-attributed definition:** an SDD with only synthesized attributes; evaluated during bottom-up (LR) parsing.
- **L-attributed definition:** an SDD where each inherited attribute depends only on the parent's inherited attributes and on siblings to the left; evaluated in one left-to-right depth-first pass (suits LL parsing).
- **Annotated parse tree:** a parse tree showing the attribute values at each node.
- **Dependency graph:** a graph showing which attributes depend on which; a topological order gives a valid evaluation order.
- **Intermediate code:** a machine-independent representation between source and target code: postfix notation, syntax trees, DAGs, three-address code.
- **Three-address code (TAC):** a sequence of instructions with at most three addresses and one operator on the right side, such as `x = y op z`.
- **Quadruple:** a record (op, arg1, arg2, result).
- **Triple:** a record (op, arg1, arg2) where results are referred to by the triple's position.
- **Indirect triples:** a list of pointers to triples, so that instructions can be reordered without changing the triples.
- **Backpatching:** generating jump instructions with empty targets and filling them in later when the targets are known (single-pass translation of Boolean and control-flow statements).

💡 Core concepts

Example SDD for a desk calculator (S-attributed)

Production	Semantic rule
L → E n	print(E.val)
E → E1 + T	E.val = E1.val + T.val
E → T	E.val = T.val
T → T1 * F	T.val = T1.val × F.val
T → F	T.val = F.val

Production	Semantic rule
F → (E)	F.val = E.val
F → digit	F.val = digit.lexval

For `3 * 5 + 4 n`, the annotated tree gives T.val = 15, E.val = 19, and prints 19.

Infix to postfix SDT: E → E + T { print('+') } | T; T → T * F { print('*') } | F; F → id { print(id.name) }. For `a + b * c` it prints `a b c * +`.

Implementation of SDT: S-attributed: keep values on the parser stack beside states (val[top]) and compute on each reduction. L-attributed: use recursive descent, passing inherited attributes as parameters and returning synthesized ones; or use marker non-terminals in LR parsing.

Forms of intermediate code

1. **Postfix notation:** operators after operands; `(a + b) * c` → `a b + c *`.
2. **Syntax tree:** interior nodes are operators, leaves are operands. A DAG shares common subexpressions.
3. **Three-address code statement types:** assignment `x = y op z`, `x = op y`; copy `x = y`; unconditional jump `goto L`; conditional jump `if x relop y goto L`; procedure call `param x`, `call p, n`, `return y`; indexed assignment `x = y[i]`, `x[i] = y`; address and pointer `x = &y`, `x = *y`, `*x = y`.

Translation of assignment statements: S → id = E { gen(id.place '=' E.place) }; E → E1 + E2 { E.place = newtemp(); gen(E.place '=' E1.place '+' E2.place) }; E → id { E.place = id.place }.

Boolean expressions

- Numerical method: true = 1, false = 0, evaluated like arithmetic.
- Short-circuit (jumping) code: `a < b or c < d and e < f` becomes jumps to true and false labels using backpatching with truelist and falselist:
- B → B1 or M B2: backpatch(B1.falselist, M.quad); B.truelist = merge(B1.truelist, B2.truelist); B.falselist = B2.falselist.
- B → B1 and M B2: backpatch(B1.truelist, M.quad); B.truelist = B2.truelist; B.falselist = merge(B1.falselist, B2.falselist).
- B → not B1: swap truelist and falselist.
- B → E1 relop E2: truelist = {nextquad}, falselist = {nextquad + 1}; gen('if' E1 relop E2 'goto _'); gen('goto _').

Flow-of-control statements

- `if (B) S1`: code for B with B.true → S1, B.false → next.
- `if (B) S1 else S2`: B; label B.true: S1; goto next; label B.false: S2; label next.
- `while (B) S1`: label begin: B; label B.true: S1; goto begin; label B.false (next).

Array references: for `A[i]` with base address `base` and element width w: address = base + i × w (with low bound low: base + (i − low) × w). Two-dimensional row-major `A[i][j]` with n2 columns: base + ((i − low1) × n2 + (j − low2)) × w.

Procedure calls: `p(x1, …, xn)` → evaluate arguments, `param x1` … `param xn`, `call p, n`.

Declarations: for each declaration, enter the name with its type and relative address (offset) in the symbol table, then increase the offset by the type's width (int 4, float 8, array of n elements n × width).

Case (switch) statements: evaluate the expression into t; then either a sequence of conditional jumps (`if t = V1 goto L1` ...), or, for many dense values, a jump table indexed by t; code for each case ends with `goto next`.

🧮 Solved examples

Example 1 (TAC, quadruples, triples). `a = b * -c + b * -c`

Three-address code:

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a  = t5
```

Quadruples:

#	op	arg1	arg2	result
0	minus	c		t1
1	*	b	t1	t2
2	minus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

Triples:

#	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Indirect triples: a statement list 35: (0), 36: (1), ..., 40: (5) pointing to the triples. Comparison: quadruples use temporaries (easy to move code, more space); triples save space but are hard to reorder; indirect triples allow reordering while saving space.

Example 2 (postfix). `(a + b) * (c - d) + e` → `a b + c d - * e +`.

Example 3 (while loop TAC). `while (i < 10) { s = s + i; i = i + 1; }`

```
100: if i < 10 goto 102
101: goto 107
102: t1 = s + i
103: s = t1
104: t2 = i + 1
105: i = t2
106: goto 100
107: (next statement)
```

Example 4 (array address). `x = A[i][j]` with A of size 10 × 20, int (w = 4), row-major, low bounds 0:

```
t1 = i * 20
t2 = t1 + j
t3 = t2 * 4
t4 = A[t3]
x  = t4
```

Example 5 (Boolean jumping code). `if (a < b || c > d) x = y + z;`

```
100: if a < b goto 104
101: goto 102
102: if c > d goto 104
103: goto 106
104: t1 = y + z
105: x = t1
106: (next)
```

Key points to remember

- Synthesized = from children; inherited = from parent and left siblings.
- S-attributed ⊂ L-attributed. S-attributed works with LR parsing.
- TAC: at most three addresses. Quadruples (op, arg1, arg2, result); triples use positions; indirect triples use a pointer list.
- Backpatching uses truelist, falselist, nextlist, merge and backpatch.

⭐ Important questions

1. Define SDD and SDT. Differentiate synthesized and inherited attributes; S-attributed and L-attributed definitions. Draw an annotated parse tree.
2. Write an SDT to convert infix expressions to postfix and show it on an example.
3. What is intermediate code? Explain postfix, syntax trees and three-address code.
4. Write quadruples, triples and indirect triples for a given expression.
5. Explain translation of Boolean expressions and flow-of-control statements with backpatching.

6. Generate three-address code for array references, procedure calls, declarations and case statements.

NEXT ENGINEER
sheerat

UNIT 4 Symbol Tables, Run-Time Administration and Error Handling

✍️ 2-mark definitions

- **Symbol table:** a data structure that stores information about identifiers (name, type, scope, size, offset, line of declaration, parameters) used by all phases.
- **Scope:** the region of a program where a declaration is valid.
- **Activation record (stack frame):** the block of memory pushed on the run-time stack for one call of a procedure.
- **Activation tree:** a tree showing how control enters and leaves procedure activations during a run.
- **Static allocation:** storage decided at compile time; does not support recursion or dynamic data.
- **Stack allocation:** activation records pushed and popped at run time; supports recursion.
- **Heap allocation:** storage allocated and freed in any order at run time (malloc/free, new/delete), for dynamic data.
- **Access link (static link):** a pointer in an activation record to the record of the lexically enclosing procedure, used to reach non-local names.
- **Control link (dynamic link):** a pointer to the activation record of the caller.
- **Display:** an array of pointers to the activation records of the currently active procedures at each nesting depth, giving faster access to non-local names than access links.
- **Error recovery:** the strategy a compiler uses to continue analysis after detecting an error, so that more errors can be reported in one run.

💡 Core concepts

Symbol table operations: insert(name, attributes), lookup(name), and for block-structured languages enter-scope and exit-scope (or set/reset).

Data structures for symbol tables

Structure	Insert	Lookup	Remarks
Linear list (unordered array or linked list)	O(1)	O(n)	Simple; slow for large programs
Ordered list / binary search	O(n)	O(log n)	Insert is costly
Binary search tree	O(log n) average	O(log n) average	Can become unbalanced
Hash table	O(1) average	O(1) average	Most commonly used; chaining for collisions

Representing scope: (1) a separate table for each scope, linked in a stack (the innermost table is searched first); or (2) one hash table where each entry has a scope number, and entries of a closed

scope are removed on block exit. Static (lexical) scope rule: a name refers to the declaration in the closest enclosing block.

Storage organization: code area | static data | stack (grows one way) | free memory | heap (grows the other way).

Activation record fields: actual parameters, returned value, control link, access link, saved machine status (return address, registers), local data, temporaries.

Simple stack allocation scheme: the caller evaluates parameters, stores the return address and old top-of-stack, and jumps; the callee saves registers and initializes local data. On return, the callee places the return value, restores the status and the stack top, and jumps back. **Calling sequence** = code for setting up the record; **return sequence** = code for restoring.

Storage allocation for block-structured languages

- Nested blocks (without procedures) can share the procedure's activation record: blocks whose lifetimes do not overlap use the same space.
- Access to non-local names under static scope uses access links (follow np – na links, where np is the nesting depth of the procedure using the name and na the depth of the one declaring it) or a display.
- Dynamic scope (as in early LISP) uses deep access (search the control-link chain) or shallow access (a central table for the current binding of each name).

Parameter passing: call by value, call by reference, copy-restore (value-result), call by name (textual substitution, Algol thunks).

Error detection and recovery

Error kind	Examples	Detected by	Recovery
Lexical	Illegal character (`@` in C code), unterminated string, identifier too long	Lexical analyzer	Delete the character, insert a missing one, replace or transpose characters; panic mode (skip until a valid token)
Syntactic	Missing semicolon, unbalanced parentheses, misspelled keyword (`whlie`)	Parser	Panic mode, phrase-level, error productions, global correction
Semantic	Type mismatch, undeclared variable, wrong number of arguments, duplicate declaration	Semantic analyzer	Report and assume a type (for example int), enter the undeclared name in the symbol table to avoid repeated messages

Syntactic recovery strategies

1. **Panic mode:** discard input symbols until a synchronizing token (such as `;` or `}`) is found. Simple, never loops forever, but skips input.
2. **Phrase level:** local correction of the remaining input, such as inserting a missing `;` or replacing `,` with `;`. Error routines can be put in empty cells of the LL or LR table.
3. **Error productions:** add productions for common mistakes to the grammar so the parser recognises them and gives a specific message.
4. **Global correction:** find the minimum number of insertions, deletions and changes to make the program valid. Theoretically best, too costly in practice.

In LL(1) parsing, synchronizing sets are usually built from FOLLOW(A). In LR parsing (YACC), the special `error` token is used: pop states until one with a transition on `error`, then discard input until a token that can follow.

🧮 Solved examples

Example 1 (activation tree). For `main` calling `fact(3)` recursively: main → fact(3) → fact(2) → fact(1) → fact(0). At the deepest point the stack holds 5 activation records (main and four fact records). Static allocation cannot handle this, because each call needs its own copy of n.

Example 2 (access links). Procedure `sort` (depth 1) contains `quicksort` (depth 2), which contains `partition` (depth 3). When `partition` uses array `a` declared in `sort`: np − na = 3 − 1 = 2, so follow **2 access links**.

Example 3 (error recovery). In `int x = 5 y = 6;` the parser finds `y` where `,` or `;` is expected. Phrase-level recovery inserts `;` and continues; panic mode would skip to the next `;`, losing `y = 6`.

Key points to remember

- Symbol table: hash table is best (O(1) average).
- Activation record fields: parameters, return value, control link, access link, saved status, locals, temporaries.
- Static allocation: no recursion. Stack: recursion. Heap: dynamic data.
- Access links: follow np − na links; display: direct index by depth.
- Recovery: panic mode, phrase level, error productions, global correction.

⭐ Important questions

1. What is a symbol table? What information does it store? Compare data structures for symbol tables. How is scope represented?
2. Explain storage organization and static, stack and heap allocation.
3. Draw and explain an activation record. Explain the calling and return sequences.
4. Explain access to non-local names using access links and displays in block-structured languages.
5. Explain lexical, syntactic and semantic errors with their detection and recovery strategies.

UNIT 5 Code Generation and Code Optimization

✍️ 2-mark definitions

- **Code generator:** the final phase; maps intermediate code to target machine instructions.
- **Basic block:** a maximal sequence of consecutive instructions with one entry (the first instruction) and one exit (the last instruction); no jumps in or out in the middle.
- **Leader:** the first instruction of a basic block.
- **Flow graph:** a directed graph whose nodes are basic blocks and whose edges show possible flow of control.
- **Next-use information:** for each name in a block, the next instruction that uses its value; used for register allocation.
- **DAG (directed acyclic graph) for a basic block:** a graph with leaves for initial values and interior nodes for operators; nodes are shared for common subexpressions.
- **Code optimization:** transformations that make code run faster or use less space without changing its meaning.
- **Machine-independent optimization:** done on intermediate code (common subexpression elimination, loop optimization).
- **Peephole optimization:** examining a short sliding window of target instructions and replacing it with shorter or faster code.
- **Common subexpression:** an expression computed earlier whose operands have not changed since.
- **Dead code:** code whose result is never used, or code that can never be reached.
- **Loop-invariant computation:** a computation inside a loop whose value does not change from iteration to iteration; it can be moved before the loop (code motion).
- **Induction variable:** a variable that changes by a fixed amount each loop iteration (for example i = i + 1; t = 4 * i).
- **Strength reduction:** replacing an expensive operation with a cheaper one (multiplication by addition, x^2 by x * x, x * 2 by x + x or a shift).
- **Data-flow analysis:** gathering information about how values flow through the program (reaching definitions, live variables, available expressions) to enable global optimization.

💡 Core concepts

Issues in the design of a code generator

1. **Input:** intermediate representation (TAC, trees, DAG) assumed free of errors, with type information.
2. **Target program:** absolute machine code, relocatable code (separate compilation, needs a linker) or assembly language (easiest).
3. **Memory management:** mapping names to addresses using symbol table offsets; labels to instruction addresses.

4. **Instruction selection:** choosing instructions; uniform and complete instruction sets make this easier; cost of each instruction matters (`INC a` vs `ADD a, #1`).
5. **Register allocation and assignment:** decide which values live in registers (allocation) and which register each uses (assignment); optimal assignment is NP-complete.
6. **Evaluation order:** some orders need fewer registers; finding the best order is NP-complete in general.

Target language and addresses: a simple target machine with instructions like `LD R, x` , `ST x, R` , `ADD R1, R2, R3` , `SUB` , `MUL` , `BR L` , `BLTZ R, L` . Addressing modes: absolute (x), register (R), indexed (a(R)), indirect (*R), immediate (#c). Instruction cost = 1 + number of extra words for memory addresses or constants.

Partitioning into basic blocks

1. Leaders: the first instruction; any target of a jump; any instruction right after a jump.
2. Each block runs from a leader up to (not including) the next leader.

Optimization of basic blocks (using DAGs): remove local common subexpressions, eliminate dead code, reorder statements, apply algebraic identities (x + 0 = x, x × 1 = x, x × 2 = x + x), constant folding (2 × 3.14 = 6.28), and copy propagation.

Value numbers and algebraic laws: each DAG node gets a value number; a hash table keyed by (op, left value number, right value number) finds an existing node in constant time, so common subexpressions are detected quickly. Commutativity (a + b = b + a) and associativity can be used to find more matches.

Simple code generator (getReg)

- Keeps a register descriptor (which names each register holds) and an address descriptor (where the current value of each name is: register, memory or both).
- For `x = y op z` : choose registers Ry, Rz (load y and z if not already in registers), choose Rx; emit `OP Rx, Ry, Rz` ; update descriptors. At block end, store live values back to memory.

Machine-independent optimizations

1. **Common subexpression elimination** (local and global).
2. **Copy propagation:** after `x = y` , use y for x where possible; often makes the copy dead.
3. **Dead code elimination.**
4. **Constant folding and constant propagation.**
5. **Loop optimizations:** - **Code motion:** move loop-invariant computations out of the loop (`while (i <= limit - 2)` → `t = limit - 2; while (i <= t)`). - **Induction variable elimination:** keep only one induction variable when several move in lock-step. - **Strength reduction:** replace `t = 4 * i` inside the loop by `t = t + 4` . - **Loop unrolling** (fewer tests) and **loop jamming/fusion** (merge two loops over the same range).

Peephole optimization techniques: remove redundant loads and stores (`ST a, R0` followed by `LD R0, a` → drop the load); remove unreachable code; flow-of-control optimization (jump to a jump → jump directly); algebraic simplification; strength reduction; use of machine idioms (auto-increment, `INC`).

Global data-flow analysis

Problem	Direction	Equations	Use
Reaching definitions	Forward	out[B] = gen[B] ∪ (in[B] − kill[B]); in[B] = ∪ out[P] over predecessors P	Constant propagation, detecting undefined use

Problem	Direction	Equations	Use
Live variables	Backward	in[B] = use[B] ∪ (out[B] − def[B]); out[B] = ∪ in[S] over successors S	Register allocation, dead code elimination
Available expressions	Forward	out[B] = e_gen[B] ∪ (in[B] − e_kill[B]); in[B] = ∩ out[P]	Global common subexpression elimination

The equations are solved iteratively until nothing changes (a fixed point). Loops in a flow graph are found using **dominators** (d dominates n if every path from the entry to n goes through d) and **back edges** (n → d where d dominates n); the natural loop of a back edge n → d is d plus all nodes that can reach n without going through d.

🧮 Solved examples

Example 1 (basic blocks and flow graph). Dot product of two 20-element arrays:

```
(1)  prod = 0
(2)  i = 1
(3)  t1 = 4 * i
(4)  t2 = a[t1]
(5)  t3 = 4 * i
(6)  t4 = b[t3]
(7)  t5 = t2 * t4
(8)  t6 = prod + t5
(9)  prod = t6
(10) t7 = i + 1
(11) i = t7
(12) if i <= 20 goto (3)
```

- Leaders: (1) (first instruction) and (3) (target of the jump). **B1 = (1)–(2), B2 = (3)–(12).**
- Flow graph: B1 → B2; B2 → B2 (back edge, the loop); B2 → exit.
- Optimizations in B2: t3 = 4 * i is a common subexpression of t1, so use t1 (b[t1]); i and t1 are induction variables; strength reduction replaces `t1 = 4 * i` with `t1 = t1 + 4` (initialize t1 = 0 in B1); then i can be eliminated by testing `if t1 <= 76 goto (3)`. Copies like `prod = t6` and `i = t7` are removed by copy propagation.

Example 2 (DAG of a basic block).

```
a = b + c
b = a - d
c = b + c
d = a - d
```

- Node n1 = b0 + c0 (label a). Node n2 = n1 − d0 (label b). Node n3 = n2 + c0 (label c). `a - d`: a is n1 and d is still d0, so it is n2 again; attach label d to n2.
- Optimized block: `a = b + c; d = a - d; c = d + c;` (b and d have the same value; if b is live after the block, add `b = d`).

Example 3 (peephole).

```
LD  R0, a
ST  a, R0        → redundant store, removed
goto L1
...
L1: goto L2      → change "goto L1" to "goto L2"
x = x * 1        → removed (algebraic identity)
y = y * 2        → y = y + y (strength reduction)
```

Example 4 (live variables). Block B: `a = b + c; d = a - e`. use[B] = {b, c, e} (used before defined), def[B] = {a, d}. If out[B] = {d, f}, then in[B] = {b, c, e} ∪ ({d, f} − {a, d}) = **{b, c, e, f}**.

Formulas to remember

- Leaders: first instruction, jump targets, instructions after jumps.
- Reaching definitions: out = gen ∪ (in − kill), in = ∪ out(pred).
- Live variables: in = use ∪ (out − def), out = ∪ in(succ).
- Available expressions: in = ∩ out(pred).
- Access links: follow np − na links.

⭐ Important questions

1. Explain the issues in the design of a code generator.
2. Define basic block and flow graph. Partition a given TAC into basic blocks and draw the flow graph.
3. Construct the DAG for a basic block and use it to optimize the block. Explain value numbering.
4. Explain machine-independent optimizations: common subexpression elimination, copy propagation, dead code elimination, constant folding.
5. Explain loop optimizations: code motion, induction variable elimination, strength reduction, loop unrolling.
6. Explain peephole optimization with examples.
7. What is global data-flow analysis? Write the data-flow equations for reaching definitions and live variables.

Last-hour checklist

☐ Phases with the `position = initial + rate * 60` example; front end vs back end; phase vs pass.

☐ Bootstrapping with T-diagrams; cross compiler.

☐ Token, lexeme, pattern; input buffering with sentinels; LEX structure; counting tokens (printf example = 10).

☐ RE → NFA → DFA → minimized DFA for (a|b)*abb (4 states).

☐ Ambiguity with id + id * id; unambiguous expression grammar.

☐ Left recursion removal; left factoring; FIRST and FOLLOW rules; LL(1) table for the expression grammar.

- ☐ Operator precedence relations; shift-reduce conflicts.
- ☐ S → CC, C → cC | d: SLR (7 states), CLR (10 states), LALR (7 states); parse cdd.
- ☐ SDD vs SDT; synthesized vs inherited; S- and L-attributed; postfix SDT.
- ☐ TAC, quadruples, triples, indirect triples for a = b * −c + b * −c.
- ☐ Backpatching rules for or, and, not; if-else and while code; array address formula.
- ☐ Symbol table structures and scope; activation record; static, stack, heap; access links and display.
- ☐ Error kinds and recovery strategies.
- ☐ Code generator issues; basic blocks and flow graph (dot product); DAG example.
- ☐ Loop optimizations; peephole; data-flow equations.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Compilers: Principles, Techniques and Tools	Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman (2nd ed. with Monica S. Lam)	Pearson	WorldCat
2	Compiler Design	K. Muneeswaran	Oxford University Press	WorldCat
3	Introduction to Compiling Techniques (2nd ed.)	J. P. Bennet	Tata McGraw Hill	WorldCat
4	Practice and Principles of Compiler Building with C	Henk Alblas, Albert Nymeyer	PHI	WorldCat
5	Principles of Compiler Design	V. Raghavan	Tata McGraw Hill	WorldCat
6	Compiler Construction: Principles and Practice	Kenneth C. Louden	Cengage Learning	WorldCat
7	Crafting a Compiler with C	Charles N. Fischer, Richard J. LeBlanc	Pearson	WorldCat

Syllabus source: AKTU B.Tech 3rd year CS/IT syllabus, 2024-25 (copy hosted by an affiliated college, PDF). The Compiler Design units follow the syllabus printed in that file (listed there as BIT052 for IT; BCS602 for CSE uses the same course content).